

ASIGNATURA

PROGRAMACION MOVIL 1

CATEDRATICO

RICARDO ENRIQUE LAGOS MENDOZA

ALUMNO

Melvin Edgardo Peña Caballero -201230010100

ASIGNACION

Tarea 2.5 Comparación práctica de Volley, OkHttp y Retrofit con manejo de fallos

FECHA

13-NOVIEMBRE-2026

Tabla de contenido

Comparación práctica de Volley, OkHttp y Retrofit ***3***
Consumo de un endpoint protegido con JWT y manejo de fallos 3
Objetivo 3
Contenido 3

Implementación del laboratorio ***4***
Volley 4
OkHttp directo 4
Retrofit y JWT 4

Metodología y tiempos medidos ***5***
Qué mide el reloj 5
Condiciones de transporte 5
Límites de interpretación 5

Cuadro comparativo · funcionamiento ***6***

Conteo, pruebas y mantenimiento ***7***
Criterio del conteo 7
Facilidad de prueba y mantenimiento 7

API detenida ***8***
Capturas del laboratorio 8
Extractos de Logcat · ComparativaHTTP 8

Modo avión sin conexión ***9***
Capturas del laboratorio 9
Extractos de Logcat · ComparativaHTTP 9

Tiempo de espera agotado ***10***
Capturas del laboratorio 10
Extractos de Logcat · ComparativaHTTP 10

Token ausente · HTTP 401 ..... ***11***
Capturas del laboratorio 11
Extractos de Logcat · ComparativaHTTP 11

Error interno · HTTP 500 ..... ***12***
Capturas del laboratorio 12
Extractos de Logcat · ComparativaHTTP 12

Recomendación final y trazabilidad ***13***
Elección para Bitácora de campo: Retrofit 13
Rama, commits y solicitud de integración 13
Fuentes de evidencia y reproducción 13

Comparación práctica de Volley, OkHttp y Retrofit

Consumo de un endpoint protegido con JWT y manejo de fallos

Objetivo

Implementar la misma consulta GET /api/registros con Volley, OkHttp directo y Retrofit; comparar su código, tiempos y comportamiento ante errores del servidor, falta de conexión, timeout y autenticación inválida o ausente.

El laboratorio forma parte de Bitácora de campo. Retrofit reutiliza el repositorio del cliente MVVM; Volley y OkHttp consumen el mismo servicio y usan el mismo JWT.

Contenido

Implementación y metodología; cuadro comparativo; cinco escenarios con quince resultados, capturas y extractos de Logcat; recomendación final y trazabilidad de Git.

Repositorio del proyecto

Resultados medidos: 15 consultas exitosas y 15 pruebas de fallos. En todos los fallos se conservó el proceso Android. La revisión del PR por otro integrante y el video son entregables separados.

TAREA 2.5 / 01

Implementación del laboratorio

ComparativaActivity ofrece tres botones y un selector de escenario. Cada solicitud muestra cantidad y duración, o un mensaje de error. Se bloquea la configuración durante la petición y se cancelan solicitudes al destruir la pantalla.

Escenario	Ruta GET	Preparación
Listado normal	/api/registros	JWT válido; mismo endpoint y datos para los tres clientes.
Tiempo de espera	/api/pruebas/demora	await Task.Delay de 15 s; timeout Android de 2000 ms.
Error interno	/api/pruebas/error	Excepción controlada, HTTP 500; solo Development.

Volley

JsonArrayRequest recibe un JSONArray y cuenta sus elementos. getHeaders envía Authorization: Bearer y Cache-Control. DefaultRetryPolicy configura el timeout y cero reintentos. La respuesta llega al hilo principal; la solicitud tiene tag y caché desactivada.

```
override fun getHeaders() = authorizationHeaders(token)
request.retryPolicy = DefaultRetryPolicy(timeout, 0,
    DefaultRetryPolicy.DEFAULT_BACKOFF_MULT)
request.setShouldCache(false)
```

OkHttp directo

Request.Builder construye GET y encabezados; enqueue ejecuta en segundo plano. response.use cierra el recurso. Gson y TypeToken convierten a List<JsonObject>. Los estados HTTP se revisan aparte de IOException; runOnUiThread publica éxito o error.

```
activeOkHttpCall = client.newCall(request)
activeOkHttpCall!!.enqueue(object : OkHttpCallback {
    // onFailure: error de transporte; onResponse: revisar HTTP y Gson.
    // runOnUiThread { mostrar resultado }
})
```

Retrofit y JWT

RegistroRepository.consultarLaboratorio delega el listado normal en listar y usa el contrato con @Url para las rutas de prueba. GsonConverterFactory produce List<RegistroResumen>. El repositorio comparte autorización y ApiResult con el CRUD; los callbacks llegan al hilo principal con la configuración Android actual.

El login local entrega un JWT de 15 minutos, conservado en memoria. Se omite Authorization al vaciar el campo para probar 401. Las rutas de fallo exigen JWT, están limitadas a Development y su selector a debug. No se incluyen tokens ni credenciales en este informe.

TAREA 2.5 / 02

Metodología y tiempos medidos

Se ejecutaron cinco rondas en orden Volley → OkHttp → Retrofit, una solicitud a la vez. Todas usaron GET /api/registros, JWT válido, timeout de 2000 ms, Pixel 4 XL emulado, app debug y API ASP.NET Core local Development en http://10.0.2.2:5000/.

Datos: 0 registros. Se comprobó que el contenido antes y después fuera idéntico mediante SHA-256 en memoria. No se modificaron datos. Wi-Fi activo, modo avión apagado; sin cambios de red durante la serie.

No hubo calentamiento del listado en este proceso; el login ocurrió antes de medir y la API ya había atendido las pruebas previas. Se conservaron las cinco muestras de cada cliente, incluida la primera.

Cliente	t1	t2	t3	t4	t5	Promedio
Volley	827	48	62	61	49	209,40
OkHttp	854	515	791	842	869	774,20
Retrofit	862	983	839	817	846	869,40

Todos los tiempos están en milisegundos. Promedio = (t1 + t2 + t3 + t4 + t5) / 5.

Qué mide el reloj

SystemClock.elapsedRealtime incluye preparación del cliente, red, conversión JSON y entrega a la interfaz. Las capturas se toman después de recibir el resultado; no se usan como cronómetro. El proceso conservó el mismo PID durante las quince consultas.

Condiciones de transporte

Volley reutiliza RequestQueue y desactiva caché y reintentos. OkHttp y Retrofit construyen cliente por consulta, sin caché de disco y con retryOnConnectionFailure(false). Los tres envían Cache-Control: no-cache, no-store. OkHttp/Retrofit no siguen redirecciones; se usó la URL final.

Límites de interpretación

Volley tuvo el menor promedio observado. La lista vacía no mide el costo de convertir registros reales. Volley cuenta un JSONArray, OkHttp convierte a JsonObject y Retrofit a DTO tipado; Retrofit además usa logging BASIC en debug. La construcción de clientes, los usos previos del servidor y el emulador impiden concluir una ventaja general de rendimiento.

El timeout de Volley se aplica al socket; en OkHttp/Retrofit se aplica a conexión, lectura y escritura. No equivale a un límite total de extremo a extremo. No se midió concurrencia ni carga.

Las 15 muestras, capturas y Logcat

TAREA 2.5 / 03

Cuadro comparativo · funcionamiento

Criterio	Volley	OkHttp directo	Retrofit
Líneas en Activity	26	42	18; delega en repositorio. No es el total.
Encabezados	getHeaders; Authorization y Cache-Control.	Request.Builder.addHeader.	@Header para JWT y @Headers para caché; sesión desde repositorio.
Conversión JSON	JsonArrayRequest entrega JSONArray; length, sin DTO.	Gson manual y TypeToken a List<JsonObject>.	GsonConverterFactory a List<RegistroResumen>.
Hilo del callback	Principal en esta cola Android.	Trabajo; requiere runOnUiThread.	Principal con configuración Android actual.
Estados HTTP	VolleyError.networkResponse. statusCode.	Response.code; no equivalen a onFailure.	Response.code y ApiResult.Failure.httpCode.
Transporte y timeout	NoConnectionError / ConnectException; TimeoutError.	ConnectException; SocketTimeoutException.	onFailure; repositorio traduce ConnectException / SocketTimeoutException.
Reintentos	DefaultRetryPolicy: 0.	retryOnConnectionFailure(false).	Igual ajuste en ApiClient (OkHttp).
Tiempo de espera	Socket por DefaultRetryPolicy.	Conexión, lectura y escritura.	Los mismos tres ajustes de OkHttp.
Caché	setShouldCache(false).	Sin caché de disco.	Sin caché de disco.
Cancelación	Tag y cancelAll al destruir.	Call.cancel al destruir.	RequestHandle.cancel al destruir.
Promedio de 5	209,40 ms	774,20 ms	869,40 ms
Fallos sin cierre	5 de 5	5 de 5	5 de 5

Comparación basada en el código de esta implementación. Los tres envían Cache-Control: no-cache, no-store. Las clases de error de las pruebas se detallan en las páginas de cada escenario.

TAREA 2.5 / 04

Conteo, pruebas y mantenimiento

Criterio del conteo

Líneas físicas no vacías, incluidas firmas y llaves. Se excluyen comentarios completos, package, imports y decoración visual. Una línea cuenta una vez aunque contenga varias instrucciones. La fila de Activity cuenta únicamente requestWithVolley, requestWithOkHttp y requestWithRetrofit, hasta antes del siguiente método.

Bloque de código	Líneas	Alcance
requestWithVolley	26	Adaptador en Activity.
requestWithOkHttp	42	Adaptador en Activity.
requestWithRetrofit	18	Adaptador; delega en el repositorio.
httpClient (Activity)	8	Auxiliar de OkHttp directo.
RegistroApi.kt completo	15	Interfaz, anotaciones y DTO; incluye CRUD y laboratorio.
ApiClient.kt completo	31	Fábrica, conversor, timeouts y logging debug.
RegistroRepository.kt completo	89	CRUD, autorización, ApiResult, RequestHandle y errores compartidos.

La cola Volley se crea en onCreate. El ciclo de vida y los auxiliares compartidos de UI, validación, tiempo y resultados quedan fuera del conteo principal. Los archivos Retrofit incluyen operaciones ajenas a este GET: no se suman como un costo mínimo equivalente del listado. Tener menos líneas en Activity refleja delegación, no menor código total.

Facilidad de prueba y mantenimiento

Cliente	Valoración sustentada en el código
Volley	Conciso para arreglos. Construcción y callbacks residen en Activity; si cambian los campos se necesita agregar mapeo/validación a objetos.
OkHttp	Request explícito y control del transporte. Requiere mantener cierre de Response, Gson, HTTP y cambios de hilo en la pantalla.
Retrofit	DTO y contrato compartidos con CRUD; errores y autorización centralizados. Para pruebas unitarias aisladas habría que inyectar el servicio: el constructor actual lo crea internamente.

Los tres se verificaron mediante UI real y API local. No se ejecutaron pruebas unitarias con servicios inyectados; la facilidad de prueba descrita es una valoración del diseño.

Reproducción: python3 scripts/count_lab_lines.py. Conteo estructurado

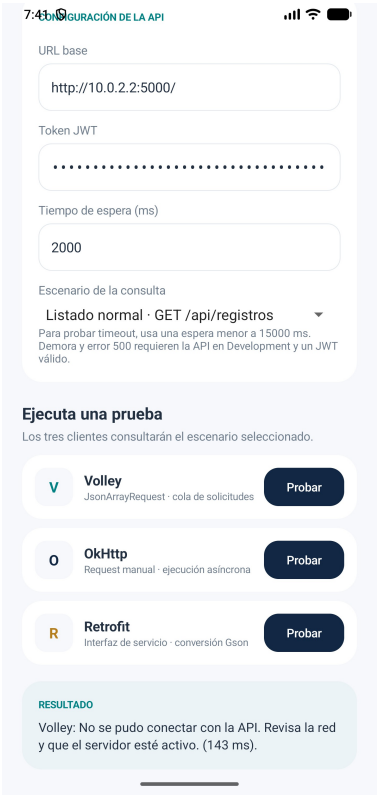
TAREA 2.5 / 05

API detenida

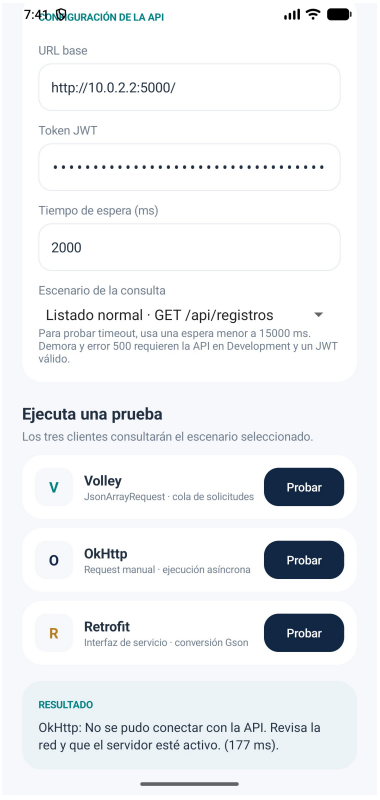
Se detuvo únicamente la API iniciada por el script, conservando la red activa. Se consultó /api/registros con JWT válido. Después se reinició la API.

Cliente	Error técnico recibido	Mensaje visible y tiempo	Sin cierre
Volley	NoConnectionError <- ConnectException	No se pudo conectar con la API. Revisa la red y que el servidor esté activo. (143 ms).	Sí
OkHttp	ConnectException <- ConnectException <- ErrnoException	No se pudo conectar con la API. Revisa la red y que el servidor esté activo. (177 ms).	Sí
Retrofit	ConnectException	Sin conexión con la API. Revisa la red y el servidor. (206 ms).	Sí

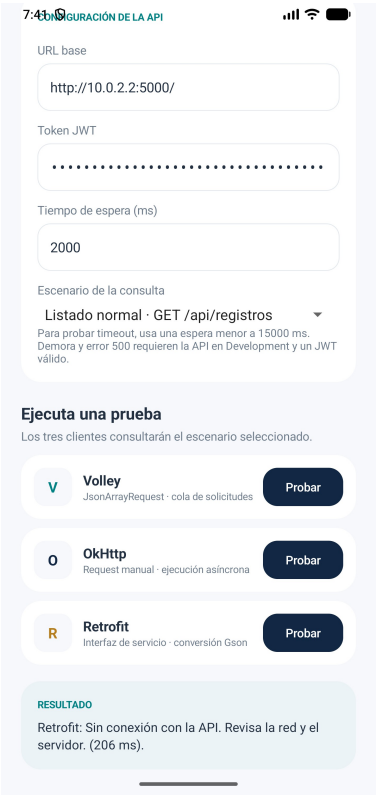
Capturas del laboratorio



Volley



OkHttp



Retrofit

Extractos de Logcat · ComparativaHTTP

Volley: NORMAL | NoConnectionError <- ConnectException | Volley: No se pudo conectar con la API. Revisa la red y que el servidor esté activo. (143 ms).

OkHttp: NORMAL | ConnectException <- ConnectException <- ErrnoException | OkHttp: No se pudo conectar con la API. Revisa la red y que el servidor esté activo. (177 ms).

Retrofit: NORMAL | ConnectException | Retrofit: Sin conexión con la API. Revisa la red y el servidor. (206 ms).

Se verificaron resultado visible, actividad de laboratorio y mismo PID del proceso. Capturas y registros completos: evidencias de los cinco fallos.

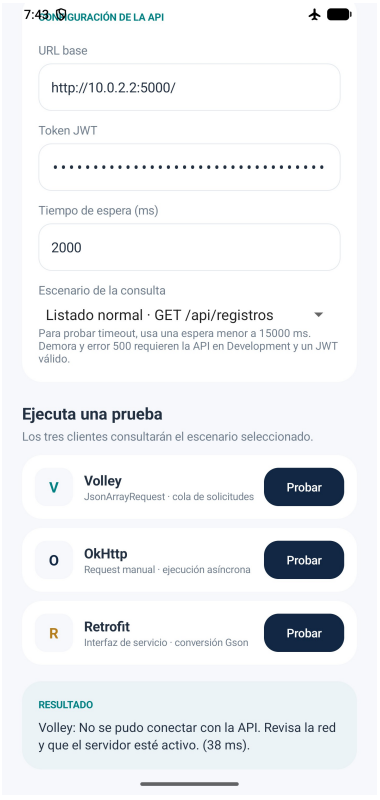
TAREA 2.5 / 06

Modo avión sin conexión

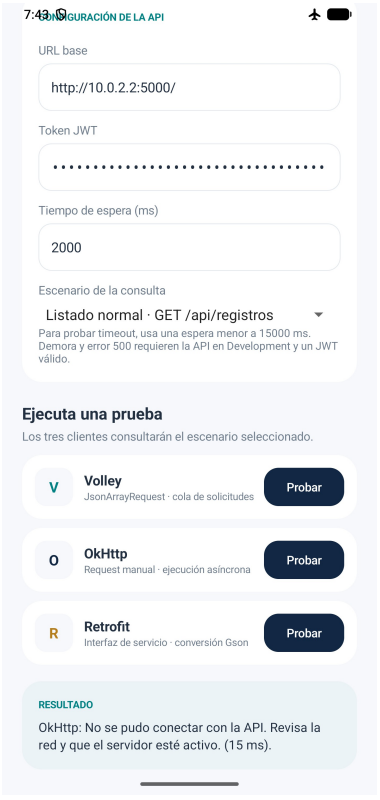
Se activó modo avión y se apagaron Wi-Fi y datos. Antes de las pruebas finales se esperó hasta que Android no tuviera red predeterminada activa. API encendida; GET /api/registros con JWT válido.

Cliente	Error técnico recibido	Mensaje visible y tiempo	Sin cierre
Volley	NoConnectionError <- ConnectException	No se pudo conectar con la API. Revisa la red y que el servidor esté activo. (38 ms).	Sí
OkHttp	ConnectException <- ConnectException <- ErrnoException	No se pudo conectar con la API. Revisa la red y que el servidor esté activo. (15 ms).	Sí
Retrofit	ConnectException	Sin conexión con la API. Revisa la red y el servidor. (37 ms).	Sí

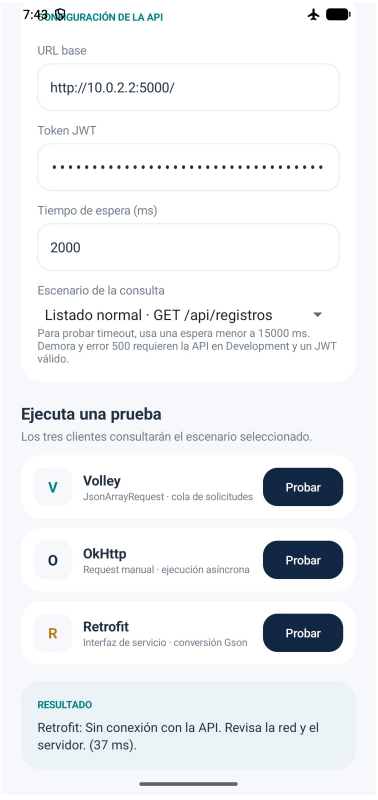
Capturas del laboratorio



Volley



OkHttp



Retrofit

Extractos de Logcat · ComparativaHTTP

Volley: NORMAL | NoConnectionError <- ConnectException | Volley: No se pudo conectar con la API. Revisa la red y que el servidor esté activo. (38 ms).

OkHttp: NORMAL | ConnectException <- ConnectException <- ErrnoException | OkHttp: No se pudo conectar con la API. Revisa la red y que el servidor esté activo. (15 ms).

Retrofit: NORMAL | ConnectException | Retrofit: Sin conexión con la API. Revisa la red y el servidor. (37 ms).

Se verificaron resultado visible, actividad de laboratorio y mismo PID del proceso. Capturas y registros completos: evidencias de los cinco fallos.

La captura inicial de red se tomó durante la transición; se repitieron los tres casos tras confirmar desconexión. Las evidencias incluidas son las finales. Al terminar se restauró la red (avión 0, Wi-Fi 1).

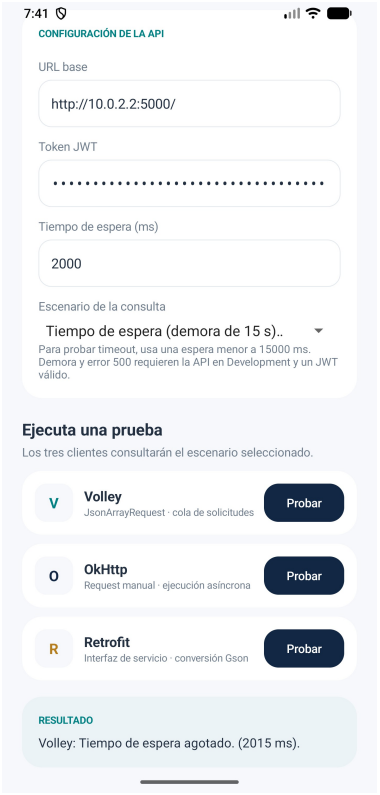
TAREA 2.5 / 07

Tiempo de espera agotado

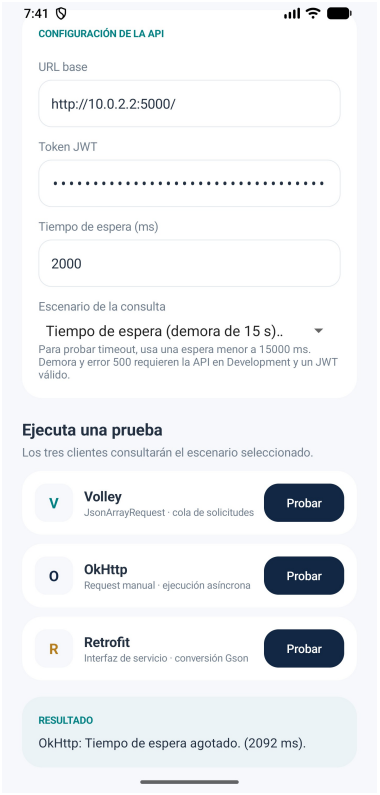
API en Development: GET /api/pruebas/demora con JWT válido. await Task.Delay espera 15 segundos y devuelve un arreglo; timeout configurado en los tres clientes: 2000 ms.

Cliente	Error técnico recibido	Mensaje visible y tiempo	Sin cierre
Volley	TimeoutError	Tiempo de espera agotado. (2015 ms).	Sí
OkHttp	SocketTimeoutException <- SocketTimeoutException	Tiempo de espera agotado. (2092 ms).	Sí
Retrofit	SocketTimeoutException	Tiempo de espera agotado. (2437 ms).	Sí

Capturas del laboratorio



Volley



OkHttp



Retrofit

Extractos de Logcat · ComparativaHTTP

Volley: DEMORA | TimeoutError | Volley: Tiempo de espera agotado. (2015 ms).

OkHttp: DEMORA | SocketTimeoutException <- SocketTimeoutException | OkHttp: Tiempo de espera agotado. (2092 ms).

Retrofit: DEMORA | SocketTimeoutException | Retrofit: Tiempo de espera agotado. (2437 ms).

Se verificaron resultado visible, actividad de laboratorio y mismo PID del proceso. Capturas y registros completos: evidencias de los cinco fallos.

TAREA 2.5 / 08

Token ausente · HTTP 401

Red restaurada y API activa. Se vació el campo JWT para omitir Authorization en GET /api/registros. El caso Android es token ausente; no se atribuye a token vencido.

Cliente	Error técnico recibido	Mensaje visible y tiempo	Sin cierre
Volley	HTTP 401	Token ausente, inválido o vencido (401). (999 ms).	Sí
OkHttp	HTTP 401	Token ausente, inválido o vencido (401). (587 ms).	Sí
Retrofit	HTTP 401	Token ausente, inválido o vencido (401). (1006 ms).	Sí

Capturas del laboratorio



Volley



OkHttp



Retrofit

Extractos de Logcat · ComparativaHTTP

Volley: NORMAL | HTTP 401 | Volley: Token ausente, inválido o vencido (401). (999 ms).

OkHttp: NORMAL | HTTP 401 | OkHttp: Token ausente, inválido o vencido (401). (587 ms).

Retrofit: NORMAL | HTTP 401 | Retrofit: Token ausente, inválido o vencido (401). (1006 ms).

Se verificaron resultado visible, actividad de laboratorio y mismo PID del proceso. Capturas y registros completos: evidencias de los cinco fallos.

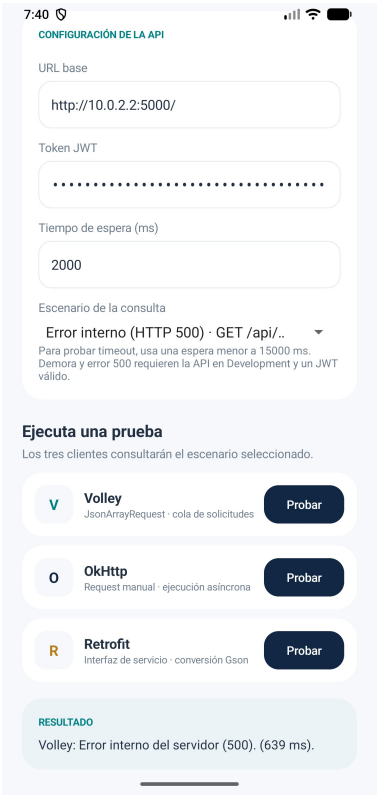
TAREA 2.5 / 09

Error interno · HTTP 500

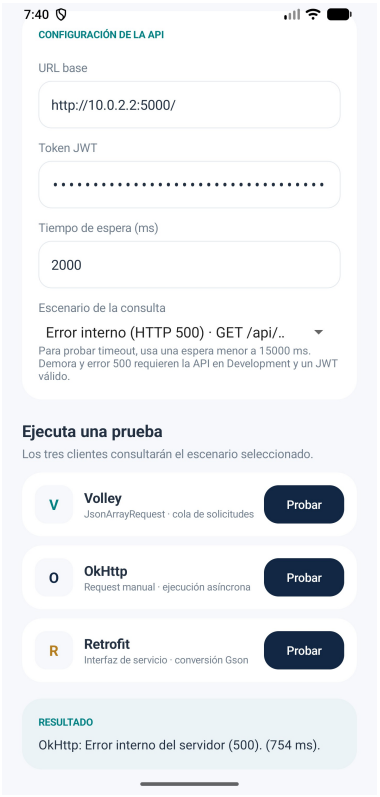
GET /api/pruebas/error con JWT válido, solo en Development. Una excepción controlada provoca HTTP 500; el manejador entrega una respuesta genérica sin detalles internos.

Cliente	Error técnico recibido	Mensaje visible y tiempo	Sin cierre
Volley	HTTP 500	Error interno del servidor (500). (639 ms).	Sí
OkHttp	HTTP 500	Error interno del servidor (500). (754 ms).	Sí
Retrofit	HTTP 500	Error interno del servidor (500). (838 ms).	Sí

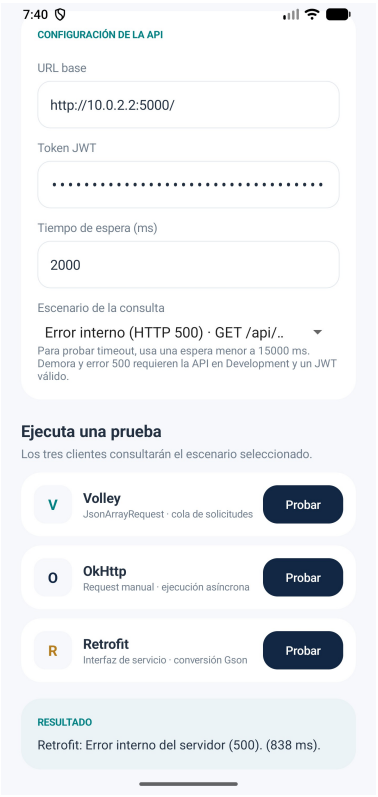
Capturas del laboratorio



Volley



OkHttp



Retrofit

Extractos de Logcat · ComparativaHTTP

Volley: ERROR | HTTP 500 | Volley: Error interno del servidor (500). (639 ms).

OkHttp: ERROR | HTTP 500 | OkHttp: Error interno del servidor (500). (754 ms).

Retrofit: ERROR | HTTP 500 | Retrofit: Error interno del servidor (500). (838 ms).

Se verificaron resultado visible, actividad de laboratorio y mismo PID del proceso. Capturas y registros completos: evidencias de los cinco fallos.

TAREA 2.5 / 10

Recomendación final y trazabilidad

Elección para Bitácora de campo: Retrofit

Recomiendo Retrofit para el proyecto final. RegistroRepository ya se comparte entre el CRUD MVVM y el laboratorio; los DTO tipados y RegistroApi concentran el contrato. Autorización, conversión y errores están centralizados, de modo que un cambio del backend requiere menos adaptación repetida en las pantallas.

Los tres clientes manejaron los cinco fallos sin cerrarse. Retrofit no fue el más rápido: promedió 869,40 ms, frente a 774,20 ms de OkHttp y 209,40 ms de Volley. La elección se fundamenta en mantenimiento y reutilización, considerando que los tiempos provienen de un listado vacío y de clientes construidos de forma diferente.

Volley resulta práctico para consultas sencillas de arreglos. OkHttp directo aporta control del transporte a cambio de más manejo manual. En este proyecto, Retrofit reduce la lógica HTTP repetida en UI y permite mantener un contrato consistente con el CRUD.

Como mejora futura convendría inyectar y reutilizar el cliente HTTP y medir con registros reales. Esas mejoras no se implementaron ni forman parte de las mediciones de este informe.

Rama, commits y solicitud de integración

Rama individual publicada: **feature/comparativa-melvin-pena**. Se conservó el historial y se incorporó main mediante fast-forward, sin force push. El PR #3 anterior ya estaba integrado; esta entrega continúa en el PR #4.

Commit	Cambio
c0ff331	Selector de escenarios compartidos en el laboratorio.
dcee07d	Quince fallos y quince mediciones con scripts y evidencias.
8654ba5	Cuadro comparativo y recomendación con resultados medidos.
2d2e3d3	Enlace de entrega y estado de publicación Git.

PR #4 hacia main · Repositorio

En la última verificación de publicación, el PR #4 estaba abierto, sin conflictos y pendiente de revisión de otro integrante. No se atribuye una revisión no realizada. El video individual de 3–4 minutos con cámara es un entregable separado. Este PDF se genera localmente para revisión del estudiante.

Fuentes de evidencia y reproducción

Quince fallos con capturas y Logcat
Quince mediciones exitosas
Informe y cuadro comparativo en Markdown

Scripts: verify_lab_failures.py, measure_lab_clients.py y count_lab_lines.py, en scripts/. Compilación e instalación debug correctas; backend compilado sin errores ni advertencias. No se modificaron registros de usuario durante estas pruebas.